

# ระบบจัดการคลังร้านอุปกรณ์การช่าง

วิชาการพัฒนาซอฟต์แวร์เชิงหลักการและการออกแบบ  
(CP353002 Principles of Software Design and Development)  
มหาวิทยาลัยวงแก่น

นำเสนอโดย

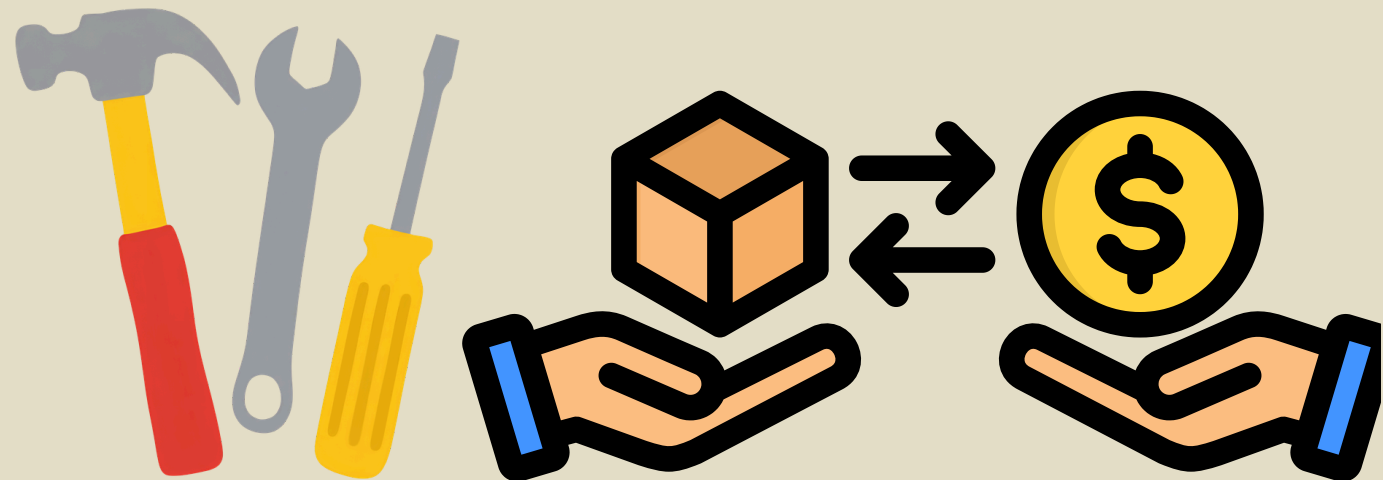
พีรพล แก้วเจริญสันติสุข (67338287-8)

พีรพัฒน์ แก่นประยุทธ์ (673380288-6)

พีสิษฐ์ ทรัพย์อุดมโชติ (673380285-2)



# ปัญหา



ร้านค้ามีการซื้อขายอุปกรณ์การช่าง



ของหายหรือขาด ไม่ตรงในคลังจริง



# กลุ่มผู้ใช้



ร้านขายอุปกรณ์การช่าง



พนักงานขาย



ผู้จัดการร้าน/Admin



# ขอบเขตของระบบ



## 1. การจัดการสินค้า

- เพิ่ม แก้ไข ลบ และค้นหาข้อมูลสินค้า
- จัดการหมวดหมู่สินค้า
- จัดการข้อมูลผู้จำหน่าย
- ค้นหาสินค้าด้วย SKU และชื่อสินค้า
- ค้นหาสินค้าตามหมวดหมู่
- รองรับ Pagination และ Sorting

## 3. การบันทึกการเคลื่อนไหวของสินค้า

- รับสินค้าเข้า (IN)
- จ่ายสินค้าออก (OUT)
- ปรับยอดสินค้า (ADJUSTMENT)
- ดูประวัติการเปลี่ยนแปลงของสินค้า
- ค้นหาประวัติตามสินค้าและช่วงเวลาโดยอัตโนมัติ
- สร้าง Stock Movement ประเภท OUT
- จัดการสถานะของ Sales Order

## 2. การจัดการคลังสินค้า

- ตรวจสอบจำนวนสินค้าคงเหลือ
- ปรับปรุงจำนวนสินค้า
- ตรวจสอบสินค้าที่มีจำนวนไม่เพียงพอ
- ตรวจสอบสินค้าที่มีจำนวนต่ำกว่าจุดสั่งซื้อขั้นต่ำ

## 4. การจัดซื้อสินค้า

- สร้างใบสั่งซื้อสินค้า
- เพิ่มรายการสินค้าในใบสั่งซื้อหลายรายการ
- ยืนยันการรับสินค้า
- เพิ่มจำนวนสินค้าเข้าสู่คลังโดยอัตโนมัติ
- สร้าง Stock Movement ประเภท IN โดยอัตโนมัติ

# ขอบเขตของระบบ



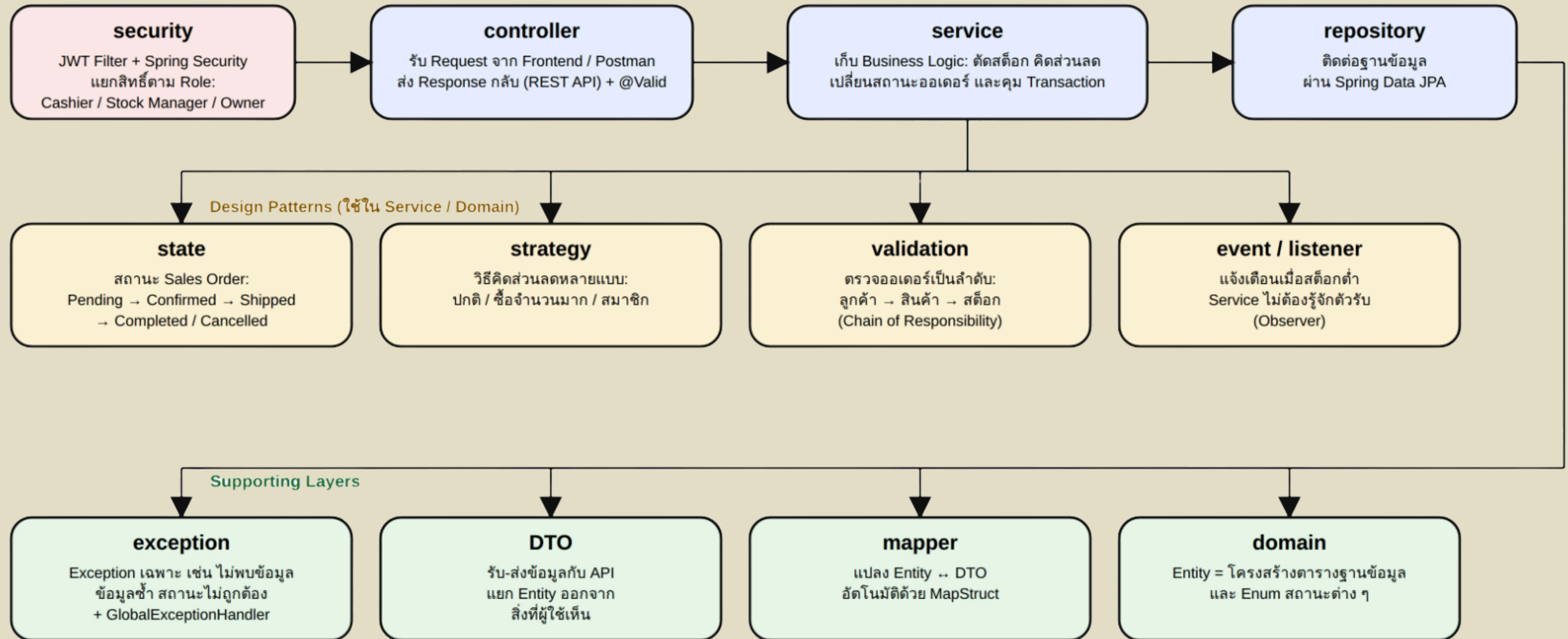
## 5. การจัดการลูกค้า

- เพิ่ม แก้ไข ลบ และค้นหาข้อมูลลูกค้า
- ตรวจสอบข้อมูลซ้ำ เช่น เบอร์โทรศัพท์หรืออีเมล

## 6. การขายสินค้า

- สร้าง Sales Order
- เพิ่มรายการสินค้าในคำสั่งซื้อหลายรายการ
- ตรวจสอบจำนวนสินค้าก่อนขาย
- ตัดจำนวนสินค้าออกจากคลังโดยอัตโนมัติ
- สร้าง Stock Movement ประเภท OUT
- จัดการสถานะของ Sales Order

# Layered Architecture



# SOLID



| หลักการ                 | สิ่งที่ทำในระบบ                                                     | ตัวอย่างคลาสที่ใช้                                                     |
|-------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------|
| S Single Responsibility | แยกการตรวจสอบออเดอร์เป็น handler ละ 1 หน้าที่                       | CustomerExistsHandler<br>ProductsExistHandler<br>StockAvailableHandler |
| O Open/Closed           | เพิ่มส่วนลดหรือสถานะใหม่ด้วยการเพิ่มคลาส ไม่แก้ if-else             | DiscountStrategy, OrderState                                           |
| L Liskov Substitution   | Strategy ทุกตัวแทนกันได้ คำนวณ BigDecimal เสมอ และจัดการ null เอง   | NormalDiscount, BulkDiscount, MemberDiscount                           |
| I Interface Segregation | Interface เล็กแยกตามโดเมน ไม่มี Fat Interface                       | ProductService, SalesOrderService, *Repository                         |
| D Dependency Inversion  | Controller และ Service ขึ้นกับ Interface ผ่าน Constructor Injection | ProductController → ProductService                                     |

# Design Patterns

## Enterprise Patterns



| Pattern              | ใช้ทำอะไร                                            | อยู่ที่ไหน                                          |
|----------------------|------------------------------------------------------|-----------------------------------------------------|
| Service Layer        | SOU business logic และ transaction ไว้ที่เดียว       | *ServiceImpl                                        |
| DTO + Mapper         | ไม่เปิด Entity ออก API และคุมรูปแบบ request/response | dto/, *Mapper (MapStruct)                           |
| Dependency Injection | Injectionลดการผูกคลาสร ทำให้ง่าย mock ตอนทดสอบได้    | Constructor Injection<br>(@RequiredArgsConstructor) |

# Design Patterns

## Enterprise Patterns



| Pattern              | ใช้ทำอะไร                                                         | อยู่ที่ไหน                                                      |
|----------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|
| Layered Architecture | แยกชั้น Controller → Service → Repository → Entity<br>ไม่ข้ามชั้น | controller/, service/, repository/, domain/                     |
| MVC                  | แยกรับ request, ข้อมูล และหน้าจอ                                  | Controller = *Controller, Model = Entity + DTO,<br>View = React |
| Repository           | ซ่อนการเข้าถึงฐานข้อมูลไว้หลัง interface                          | ProductRepository (Spring Data JPA)                             |

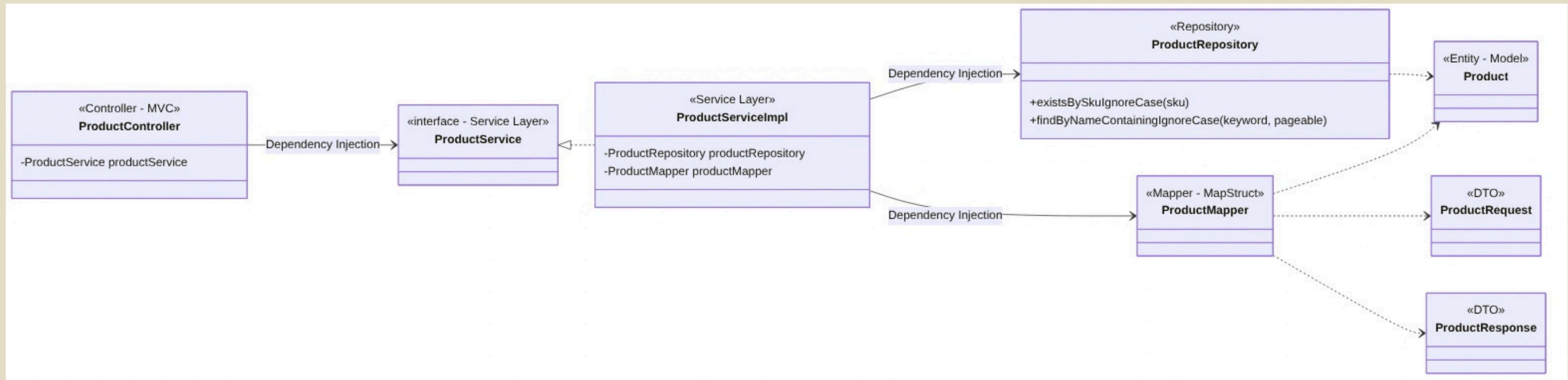
# Design Patterns

## GoF Patterns

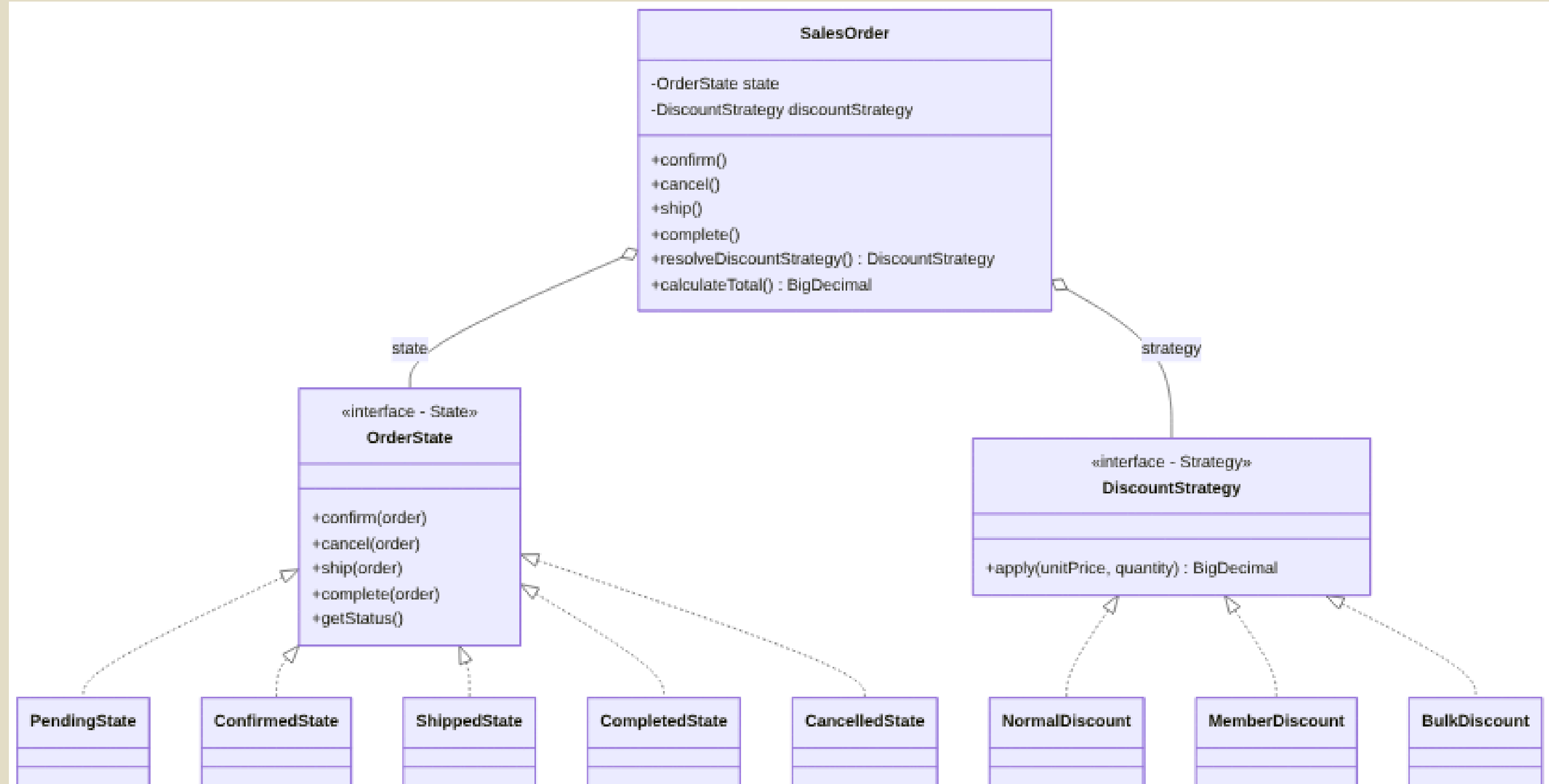


| Pattern                 | ใช้ทำอะไร                                                 | อยู่ที่ไหน                                                                                                                  |
|-------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| State                   | ใบสั่งขายมี 5 สถานะ แต่ละสถานะทำได้ไม่เหมือนกัน           | SalesOrder ส่งต่อไปยัง OrderState (Pending, Confirmed, Shipped, Completed, Cancelled) ทำรายการไม่ได้จะโยน exception ตอบ 409 |
| Strategy                | คิดราคาต่างกันตามลูกค้าและจำนวน                           | DiscountStrategy มี Normal, Member (ลด 10%), Bulk (ซื้อ $\geq 10$ ชิ้น ลด 10%)                                              |
| Chain of Responsibility | ตรวจใบสั่งขายหลายชั้นโดยแต่ละชั้นเป็นคลาสเดียว            | CustomerExists $\rightarrow$ ProductsExist $\rightarrow$ StockAvailable ชั้นไหนไม่ผ่านก็หยุดทันที                           |
| Chain of Responsibility | สต็อกต่ำต้องแจ้งเตือน โดย Service ไม่ต้องรู้ว่าใครรับแจ้ง | StockMovementServiceImpl ส่ง LowStockEvent แล้ว LowStockAlertListener รับไปแจ้งเตือน                                        |

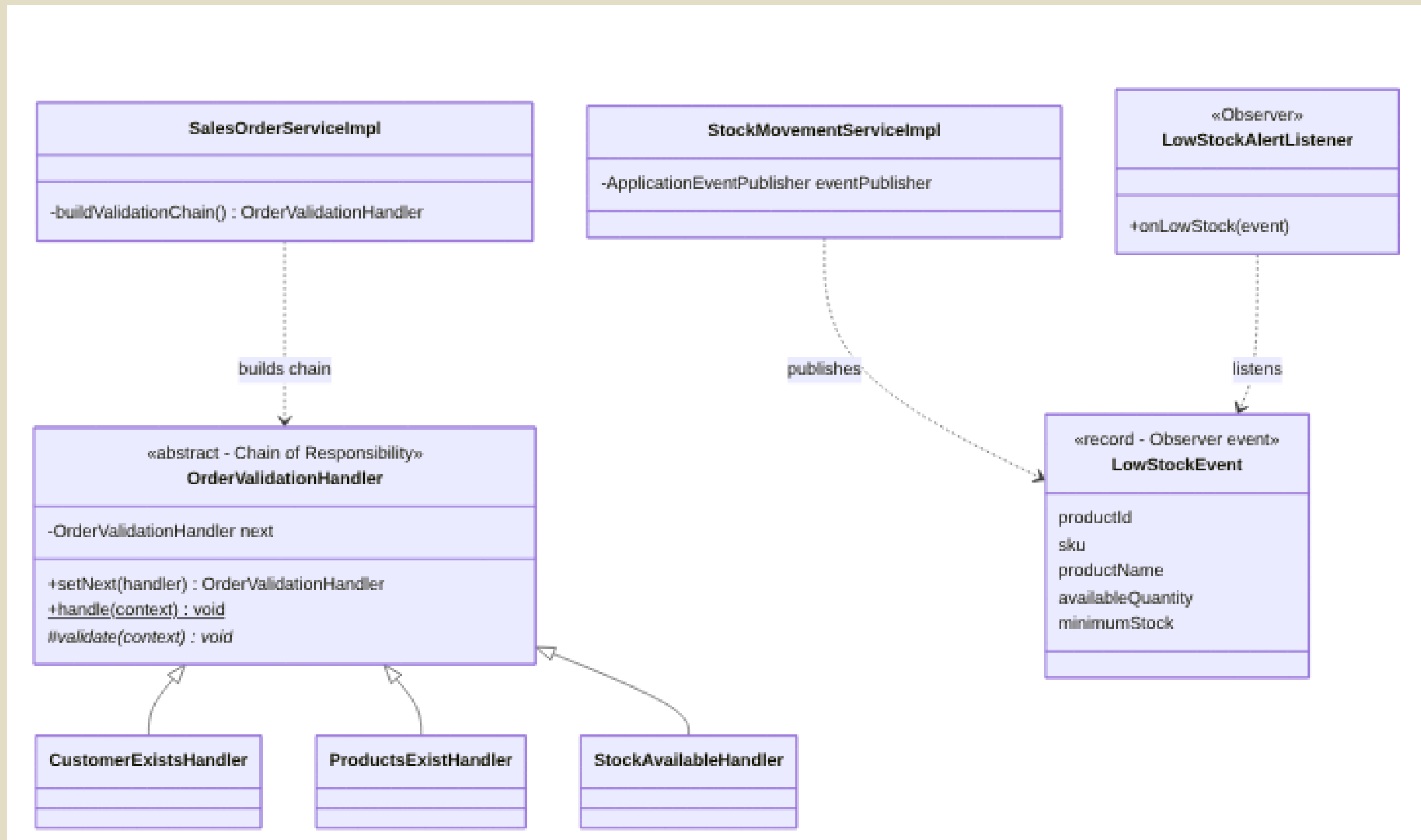
# Class Diagram



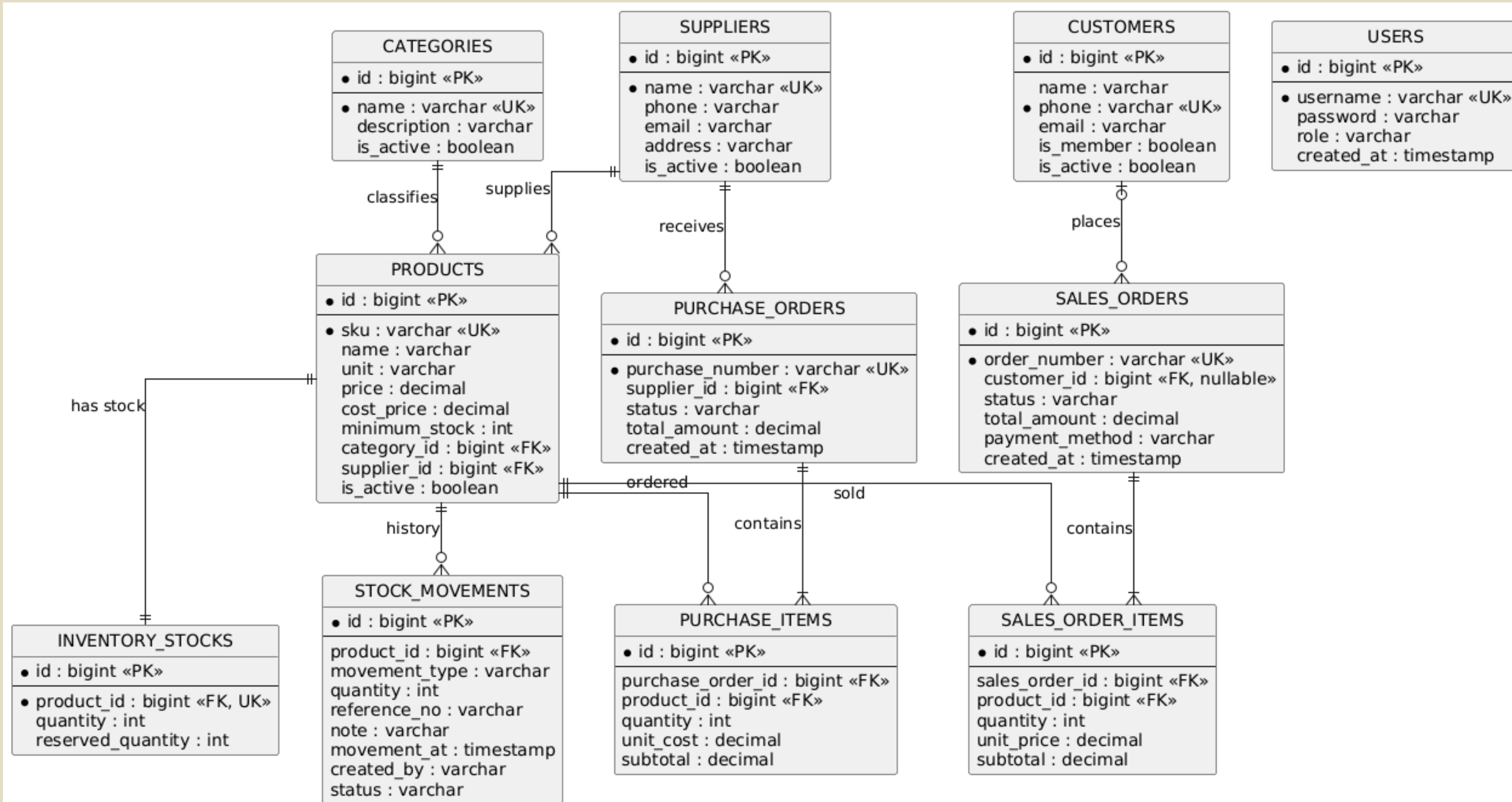
# Class Diagram



# Class Diagram



# ER Diagram



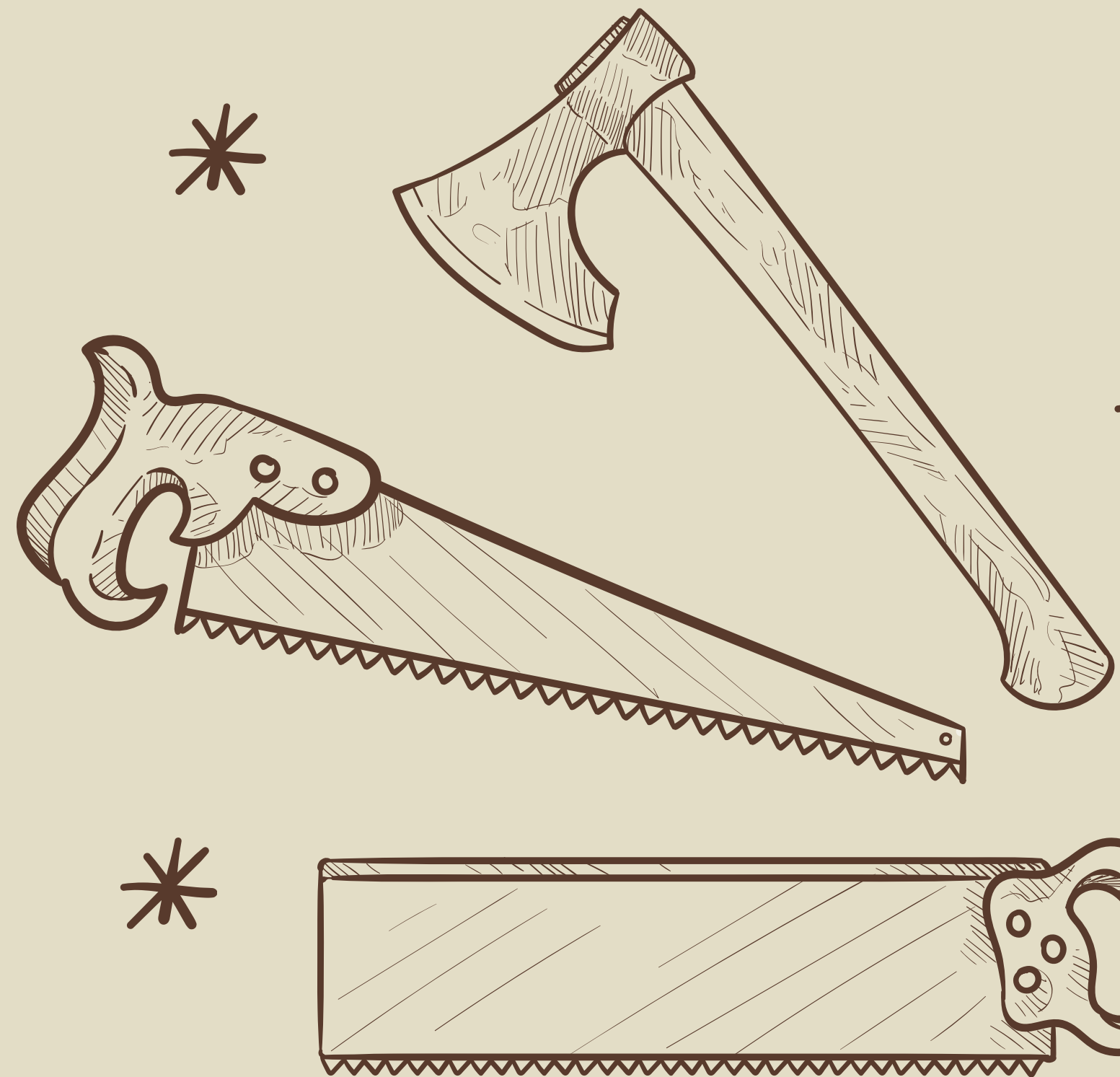


# Demo

# ปัญหาที่พบและแนวทางปรับปรุง



| ปัญหาที่พบ                                             | แนวทางปรับปรุง                                                   |
|--------------------------------------------------------|------------------------------------------------------------------|
| ตรวจสอบใบสั่งขายหลายเงื่อนไขในเมธอดเดียวจนยาวและแก้ยาก | แยกเป็น Chain of Responsibility 3 ชั้น (ลูกค้า → สินค้า → สต็อก) |
| สถานะใบสั่งขาย 5 สถานะ ทำให้ if-else ซ้อนกันหลายชั้น   | ใช้ State Pattern ให้แต่ละสถานะกำหนดเองว่าเปลี่ยนไปไหนได้        |
| Client ส่งราคาเองได้ จึงมีความเสี่ยงโกงราคา            | เซิร์ฟเวอร์ใช้ products.price เสมอ request ไม่มีฟิลด์ราคา        |



\* THANK YOU  
Q/A